

Roteiro de Apresentação - Compilador GYH

Dicas para a Gravação

- Tempo mínimo exigido da dupla: 10 minutos
 - Tenha os arquivos `programa_lexico.gyh`, `programa6.gyh` e `programa2.gyh` abertos no editor antes de começar.
-

[00:00 - 01:30] Apresentação e Arquitetura Léxica

Ação: Mostrar o arquivo `GyhGrammar.g4` (dê destaque nas regras do final - Tokens).

Felipe: "Olá, professora. Eu sou o Felipe Rodrigues e junto com o Guilherme desenvolvemos o Compilador da linguagem GYH. Eu vou focar na explicação da análise léxica, sintática e semântica. Começando pelo léxico, nós usamos o ANTLR para mapear todas as nossas Expressões Regulares em Tokens. Se rolarmos a tela aqui pro final do `GyhGrammar.g4`, verão que criamos os identificadores base como `NumInt`, `Var`, e utilizamos a instrução `-> skip` no WS para limpar automaticamente espaços e tabulações da nossa árvore sintática."

[01:30 - 03:00] Arquitetura Sintática

Ação: Rolar a tela para o bloco das regras de `programa` e comando no `GyhGrammar.g4`.

Felipe: "Para o Analisador Sintático, garantimos a estrutura exigida, dividindo o código na sessão DEC (onde mapeamos as variáveis locais) e a sessão PROG, onde caem os comandos operacionais em si. Nossa gramática valida desde atribuições até loops estruturados e trata naturalmente regras com recursividade à esquerda graças ao ANTLR v4."

[03:00 - 05:00] Arquitetura Semântica

Ação: Rolar para o topo do `.g4` e abrir também as classes `SymbolTable.java` e `Symbol.java`.

Felipe: "Na parte Semântica, nós utilizamos o bloco especial `@members` do ANTLR para acoplar instâncias Java puras junto da varredura da árvore. Construímos uma classe `SymbolTable`, funcionando através de um `HashMap`. Eu desenvolvi validações como `verificaUsoVariavel` (para impedir que uma variável seja usada sem existir) que disparam `RuntimeExceptions` impedindo a continuidade do Parser em caso de erro."

[05:00 - 07:00] Demonstração das 3 Fases de Erro no Terminal

Ação 1: Rodar o compilador no `programa_lexico.gyh`. **Felipe:** "Validando o Léxico: Ao rodar o `programa_lexico.gyh`, injetei um arroba @ solto na atribuição. O listener captura na hora e joga um *token recognition error*."

Ação 2: Rodar o compilador no `programa6.gyh`. **Felipe:** "Para o Sintático: No `programa6`,

a lógica tenta fazer uma atribuição de variável utilizando um duplo igual (==). O analisador sintático barra com *no viable alternative at input 'fatorial== '* alertando a quebra grave da árvore de sintaxe."

Ação 3: Rodar o compilador no `programa2.gyh`. **Felipe:** "E por fim a Semântica! No `programa2.gyh`, durante uma validação de SE, há o uso de uma variável `num4` que jamais foi declarada no cabeçalho. A minha tabela intercepta em Java e expõe a mensagem clara: *Erro Semântico: Variável não declarada! -> num4.*"

Ação Final: Transição. **Felipe:** "Como vimos, nossa estrutura blindou todos os erros de entrada perfeitamente. Agora, eu passo a palavra para o Guilherme, que vai explicar a geração de código C e rodar os programas válidos."